Prompt: can you write me a C++ program to take in a file taking in a sudoku board with k squares filled and use recursive brute-force to find every combination possible to fill it out.

the input files are structued like this:
5 _ _ _ _ _ 1 7 _
1 _ 6 5 _ 9 _ 4 _
4 7 2 1 _ 6 _ _ _
9 _ _ _ _ _ 5 _ _
_ 1 8 _ 9 5 4 _ _
6 _ _ 4 _ 2 3 8 9
_ 4 _ _ _ _ 9 3 _
_ 9 _ 7 _ 3 _ 5 _
2 6 3 9 5 8 7 1 4

I will explain what needs to be achieved in steps:

1. reading the file - the program should first ask the user for a .txt file containing the sudoku grid with k numbers filled. We will treat the sudoku board as a matrix with length and width of 9 (the standard sudoku board size).

2. solving the file - we want to use a recursive function that solves for every possible combination to solve the sudoku board. the function should start at the first empty block and check for numbers 1 to 9. if a number does not appear in the same row, column, or grid box, insert it into the space and go to the next empty square to repeat. this should go on until we reach the final empty box. if there are no possible values to insert in a box, the function should backtrack to the previous box to try another candidate. this should repeat until all boxes are filled (k=81) or no solution exists for all combinations tried.

3. printing - each possible combination where every box is filled (k=9) should be printed in the same form as the input file. if no possible combination is found, the program needs to print that a solution is not possible.

Gemini Code (prompted online on personal computer (gemini pro)):

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
```

```
const int SIZE = 9;

// Function to print the Sudoku board in the required format
void printBoard(const std::vector<std::vector<char>>& board) {
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            std::cout << board[r][c] << (c == SIZE - 1 ? "" : " ");
        }
        std::cout << "\n";
    }
    std::cout << "\n";
}

// Function to check if placing a digit at board[row][col] is valid
bool isValid(const std::vector<std::vector<char>>& board, int row, int col, char val) {
    // Check row and column constraints
    for (int i = 0; i < SIZE; ++i) {
        if (board[row][i] == val) return false;
        if (board[i][col] == val) return false;
    }

    // Check the local 3x3 grid box constraint
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;
    for (int r = startRow; r < startRow + 3; ++r) {
        for (int c = startCol; c < startCol + 3; ++c) {
            if (board[r][c] == val) return false;
        }
    }

    return true;
}

// Recursive function to find and print all possible combinations
void solveSudoku(std::vector<std::vector<char>>& board, int row, int col, int& solutionCount) {
    // If we've reached the end of the row, move to the next row
    if (col == SIZE) {
        row++;
        col = 0;
    }

    // Base case: If we successfully fill all rows, we found a valid solution
    if (row == SIZE) {
```

```
        solutionCount++;
        std::cout << "--- Solution #" << solutionCount << " ---\n";
        printBoard(board);
        return;
    }

    // Skip pre-filled blocks
    if (board[row][col] != '_') {
        solveSudoku(board, row, col + 1, solutionCount);
        return;
    }

    // Recursive brute-force step: try numbers '1' through '9'
    for (char val = '1'; val <= '9'; ++val) {
        if (isValid(board, row, col, val)) {
            board[row][col] = val; // Tentatively place value

            // Recurse to the next cell
            solveSudoku(board, row, col + 1, solutionCount);

            board[row][col] = '_'; // Backtrack
        }
    }
}

int main() {
    std::string filename;
    std::cout << "Enter the name of the Sudoku file (e.g., puzzle.txt): ";
    std::cin >> filename;

    std::ifstream infile(filename);
    if (!infile) {
        std::cerr << "Error: Could not open file " << filename << "\n";
        return 1;
    }

    std::vector<std::vector<char>> board(SIZE, std::vector<char>(SIZE));

    // Read the grid from the file
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            if (!(infile >> board[r][c])) {
                std::cerr << "Error: Invalid file format. Expected a 9x9 grid.\n";
                return 1;
```

```
            }
        }
    }
    infile.close();

    int solutionCount = 0;
    solveSudoku(board, 0, 0, solutionCount);

    // If no valid combination filled the board, state that a solution is not possible
    if (solutionCount == 0) {
        std::cout << "A solution is not possible for this board configuration.\n";
    } else {
        std::cout << "Total solutions found: " << solutionCount << "\n";
    }

    return 0;
}
```

Chatgpt's code (also prompted on their website, on personal computer):

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

const int SIZE = 9;
using Board = vector<vector<int>>;

// Check whether placing num at (row, col) is valid.
bool isSafe(const Board& board, int row, int col, int num) {
    // Check the row and column.
    for (int i = 0; i < SIZE; i++) {
        if (board[row][i] == num) {
            return false;
        }

        if (board[i][col] == num) {
            return false;
        }
    }
```

```
    // Find the top-left corner of the 3x3 box.
    int boxRow = (row / 3) * 3;
    int boxCol = (col / 3) * 3;

    // Check the 3x3 box.
    for (int r = boxRow; r < boxRow + 3; r++) {
        for (int c = boxCol; c < boxCol + 3; c++) {
            if (board[r][c] == num) {
                return false;
            }
        }
    }

    return true;
}

// Print the board in the same format as the input file.
void printBoard(const Board& board) {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (col > 0) {
                cout << " ";
            }

            if (board[row][col] == 0) {
                cout << "_";
            } else {
                cout << board[row][col];
            }
        }
        cout << '\n';
    }

    cout << '\n';
}

// Recursively find and print every valid solution.
// Returns the number of solutions found in this branch.
long long solveSudoku(Board& board) {
```

```
    // Find the first empty square.
    int row = -1;
    int col = -1;

    for (int r = 0; r < SIZE; r++) {
        for (int c = 0; c < SIZE; c++) {
            if (board[r][c] == 0) {
                row = r;
                col = c;
                break;
            }
        }

        if (row != -1) {
            break;
        }
    }

    // Base case: there are no empty squares.
    if (row == -1) {
        printBoard(board);
        return 1;
    }

    long long solutions = 0;

    // Try every possible value for this square.
    for (int num = 1; num <= 9; num++) {
        if (isSafe(board, row, col, num)) {
            // Place the candidate.
            board[row][col] = num;

            // Recursively solve the rest of the board.
            solutions += solveSudoku(board);

            // Backtrack to explore other candidates.
            board[row][col] = 0;
        }
    }
```

```
    return solutions;
}

// Check that the original clues do not conflict.
bool validInitialBoard(const Board& board) {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            int num = board[row][col];

            if (num != 0) {
                // Temporarily remove the clue so isSafe()
                // checks for other copies of the same number.
                Board& mutableBoard =
                    const_cast<Board&>(board);

                mutableBoard[row][col] = 0;
                bool safe = isSafe(board, row, col, num);
                mutableBoard[row][col] = num;

                if (!safe) {
                    return false;
                }
            }
        }
    }

    return true;
}

int main() {
    string filename;

    cout << "Enter the path to the Sudoku .txt file: ";
    getline(cin, filename);

    ifstream inputFile(filename);

    if (!inputFile.is_open()) {
        cerr << "Error: Could not open file: "
             << filename << '\n';
```

```
    return 1;
}

Board board(SIZE, vector<int>(SIZE, 0));

// Read exactly 81 tokens.
string token;
int count = 0;

while (inputFile >> token) {
    if (count >= SIZE * SIZE) {
        cerr << "Error: The file contains more than "
             << "81 squares.\n";
        return 1;
    }

    if (token == "_") {
        board[count / SIZE][count % SIZE] = 0;
    } else if (token.size() == 1 &&
               token[0] >= '1' &&
               token[0] <= '9') {
        board[count / SIZE][count % SIZE] =
            token[0] - '0';
    } else {
        cerr << "Error: Invalid token: "
             << token << '\n';
        return 1;
    }

    count++;
}

inputFile.close();

if (count != SIZE * SIZE) {
    cerr << "Error: Expected 81 squares, but found "
         << count << ".\n";
    return 1;
}
```

```
    if (!validInitialBoard(board)) {
        cout << "No solution is possible: "
             << "the initial clues conflict.\n";
        return 0;
    }

    cout << "\nSearching for all possible solutions...\n\n";

    long long totalSolutions = solveSudoku(board);

    if (totalSolutions == 0) {
        cout << "No solution is possible.\n";
    } else {
        cout << "Total solutions found: "
             << totalSolutions << '\n';
    }

    return 0;
}
```

COMPARISON:

Notable comparisons (code structure)

	Gemini	Chatgpt
solvesudoku function	one notable change is that gemini's function is a void function as he global solving counter is a global int that is increased without having to return it in the function. the functions recursive statement is just calling the function again at the next slot.	chatgpt's function is a "long long" function which is capable of returning large whole numbers which is supposedly in the case that the board inputted is near empty. the variable in mind is the solution counter which is how the recursion works (the line solutions += solveSudoku(board); which i feel is a bit unnecessary)
solvesudoku function, continued	gemini and chatgpt's structure is somewhat similar though the variables differ. gemini goes through each and every section of the matrix and skips based on	chatgpt's structure is different from geminis by using a for loop to find the next blank item in a list. in the case of recursion this saves a good chunk of space.

	if statements (if it is blank go to the next one, if it is the end of the row go to the next one, etc).	
original puzzle checking	gemini does not do initial sudoku board checking for validity. this means that the system will continue filling spaces until it reaches a point where it cannot add any valid points even after recursing. This takes up more time in the case that a board is noticeably unsolveable from the beginning, but may not be necessary otherwise.	chatgpt does check if the initial sudoku board is valid to start which is helpful for saving time in certain situations where a board is nearly empty yet has a small error initially.
the board	gemini's board is a matrix called board	chatgpt's board is also a matrix but is inside a board object that is used in the functions. not necessarily an important difference, but the change was notable enough in my personal opinion
space complexity	we use a vector to store the values and it is a fixed 9x9 board, but also need memory for the functions in the stack of recursion which can only be as big as our board, so its O(n) with a max of n=81 (also can be considered O(1))	same as gemini but the while loop lowers the amount of recursions by the number of initially filled spaces
time complexity	when solving the board we try 9 options for as many spaces as we can, which in the **worse case possible** would be all 81, so O(9^n) where n<=81	same as gemini, though there are better safeguards like preventing it from starting if the board is already invalid.
comments	gemini does a little bit more to explain chunks of code, even if it is minimal, the general idea of each chunk of lines is written down.	chatgpt does give some comments, but leaves out a lot on certain places that may seem like theyre "easy to grasp" or not what was specifically asked.
extra		i dont really know why it was important for chatgpt to change underscores in the matrix into 0's. i feel like its unnecessary

CHOICE: Gemini prompt

reasoning: more agreeable code formatting. local solution counter unneeded. chatgpt has functions that should be incorporated though, like validity checking first.
-overall, the two codes are quite similar outside of mixing things around and a few different handling cases. There weren't many striking errors this time.

- **Modifying code (what do i plan on changing?)**
    - adding initial matrix checking which may help time complexity.
    -
    - adding while loop to skip unnecessary recursions
    - (NOTE: adding it in took a couple different angles. at first i tried skipping to the next value, but ended up having some out of bounds issues. then, when i got it to work, the program printed the same matrix multiple times.
    -
    - Comments - a better idea of what each function and block of code does is essential in keeping maintainability. Gemini's prompt gives a good start, but could always be more descriptive. ✅

CHANGED CODE:

```
//Name of program: CW_SudokuSolver.cpp
//Discription: A c++ program that takes in a .txt file with an unfinished
sudoku board
//and uses a recursive function to brute-force every possible combination.
//
//Inputs: The inputted text file should be nine lines filled with
// nine characters (1-9) or an underscore (_) which represents an empty
square.
//
//Outputs: the program will print every possible combination
// as well as the amount of possible solutions to the terminal.
//
// collaborators: none
//other sources: Google Gemini's code, chatgpt's code for pros and cons
// as well as some improvements
//Author: Christopher Warren
//Creation date: 10/15/2026
// Revision date: 10/15/2026
// Revisions:1. added comments. 2.included validity checker. 3.
implemented while loop to skip unnecessary recursions. 4. restructured
```

```
solvesudoku function to find empty spaces before calling recursive
function



#include <iostream> // input/output
#include <fstream> // reading/writing files
#include <vector> //vectors (for board)
#include <string> //strings

const int SIZE = 9; // out sudoku board will always be a 9x9, so we keep
it constant here

// Function Print Board
//  prints the Sudoku board in the same format as it was input
// inputs: our coard matrix
// outputs: does not return anything but prints the board out
void printBoard(const std::vector<std::vector<char>>& board) {
    for (int r = 0; r < SIZE; ++r) { //goes through rows of matrix
        for (int c = 0; c < SIZE; ++c) { //goes through columns of matrix
            std::cout << board[r][c] << (c == SIZE - 1 ? "" : " ");
//prints value, no extra space at last columnn
        } // end of column for loop
        std::cout << "\n"; //new line for new row
    } // end of row for loop
    std::cout << "\n"; // we want to separate the matrix from the next
print here
} // end of printBoard Function

// Boolean Function isValid
// Function to check if placing a digit at board[row][col] is valid
//(valid meaning it:
//      1. does not equal a same value in the same row
//      2. does not equal a same value in the same column
//      3. does not equal a same value in the same 3x3 grid
// input: board, row and col value, and digit
// output: if it is valid, returns True.
//         if it is invalid, returns False.
bool isValid(const std::vector<std::vector<char>>& board, int row, int
col, char val) {
    // for the length and width of the board
```

```
    for (int i = 0; i < SIZE; ++i) {
        if (board[row][i] == val) return false; //checks rows for a
similar value
        if (board[i][col] == val) return false; // checks columns for a
similar value
    } // end of for loop

    // Check the local 3x3 grid box constraint
    // our int row divided by three gives an int value
    //that is always rounded down,
    // so multiplying by 3
    //gives either 0, 3, or 6 - starting rows/columns
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    //our rows and columns for loops now only check the 3x3 grid
    //starting where those values are
    for (int r = startRow; r < startRow + 3; ++r) {
        for (int c = startCol; c < startCol + 3; ++c) {
            if (board[r][c] == val) return false; //check 3x3 grid for
similar value.
        }// end of if statement
    }// end of for statement

    return true; // if there are no similar values already put, our digit
is valid.
} // end of isValid function

// void Function solveSudoku
// Recursive function to find and print all possible combinations
// recursively checks every index in the matrix, calling the function for
the next
// if it is not empty already,
// and then trying all possible candidates for that box, checking if
valid,
//and calling the funtion for the next index
//
//inputs: the board, the current row and column index, the global solution
count
```

```
//outputs: none, but when we fill all slots we call the printBoard
function.
// Additionally, when we find a solution, we increase the solution count
variable,
// so when it is over we will also know the amount of solutions.
void solveSudoku(std::vector<std::vector<char>>& board, int row, int col,
int& solutionCount) {

    //try numbers '1' - '9' in this index
    // we use a for loop to check all values
    // if a number can be placed, input it it, call the function again,
    // and then remove it for the next number
    for (char val = '1'; val <= '9'; ++val) {
        if (isValid(board, row, col, val)) {
            board[row][col] = val; // Tentatively place value

            //update 3: we find the next empty value first before
recursing
            //nextrow and nextcol are the values we will recurse to next
            int nextrow = row;
            int nextcol = col + 1;
            bool empty_exists = false; //this checks if the board is
alreaxdy complete

            while (nextrow<SIZE){ // while we arent at the end of the
board

                if (nextcol == SIZE){ // if we are at the end of the row
first
                    nextrow = nextrow+1; //move to next row
                    nextcol = 0; //first item in row
                    continue; //skip the rest
                }//end of if statement

                if (board[nextrow][nextcol]=='_' and nextrow<SIZE){ //if
we find an empty space
                    empty_exists = true; //keep track
                    break; //end while loop
                }//end of if statement
                nextcol = nextcol+1; //next index in row
```

```
            }//end of while loop

            // call function for next index if an empty exists
            if (empty_exists){
                solveSudoku(board, nextrow, nextcol, solutionCount);
            }else{ //else, the board is full, so we print it out
                solutionCount++; // increase the solution counter
                std::cout << "--- Solution #" << solutionCount << "
---\n"; // prints current attempt (left in cause i think its helpful)
                printBoard(board); //function to print board
            } // end of if else statement
            board[row][col] = '_'; // Backtrack
        } // end of if statement
    }// end of for loop
} // end of solveSudoku function

//boolean function validboard
//checks if the initial board is valid to start.
// if the board isnt solveable form the beginning, we can save lots of
time by checking beforehand.
//inputs: the board matrix
//outputs; if the board has no invalid numbers, return True
//         if the board has an invalid number, return False
bool validboard(std::vector<std::vector<char>>&board){
    char temp = '_';
    for (int row = 0; row < SIZE; ++row) { //go through rows
        for (int col = 0; col < SIZE; ++col){ // go through columns
            if (board[row][col] != '_'){ // if we find an index that isnt
empty
                temp = board[row][col]; //move the value to a temporary
variable
                board[row][col] = '_'; //change the index value to a blank
                if (!isValid(board, row, col, temp)){ // if the value is
invalid there initially
                    return false; // the board doesnt have any solutions.
                } //end of if statement
                //otherwise return the value and continue
                board[row][col] = temp;
            }// end of if statement
        }//end of column for loop
```

```
    }//end of row for loop
    return true; // if all digits are valid return true
}//end of validboard function


//main function
//takes in .txt file and checks it's able to be read and the right size
// before moving it to the solve sudoku function
// no initial inputs but asks for .txt file in folder.
// will end program if issues were found with the file.
// will call sudoku solver function which will output all possible sudoku
combinations
//of the given unfinished board
// as well as the number of possible solutions.

int main() {
    std::string filename; // var for user file
    std::cout << "Enter the name of the Sudoku file (e.g., puzzle.txt): ";
    std::cin >> filename; //put in the user file given

    //check if file can be opened
    std::ifstream infile(filename);
    if (!infile) { // if it cant
        std::cerr << "Error: Could not open file " << filename << "\n";
//print to terminal
        return 1; // end program
    }

    //print our board variable
    std::vector<std::vector<char>> board(SIZE, std::vector<char>(SIZE));

    // Read the grid from the file
    for (int r = 0; r < SIZE; ++r) { // row for loop
        for (int c = 0; c < SIZE; ++c) { //column for loop
            if (!(infile >> board[r][c])) {//if there are less characters
than the board expected:
                std::cerr << "Error: Invalid file format. Expected a 9x9
grid.\n"; //tell user
                return 1;// end program
            }
```

```
        }
    }
    infile.close(); // close file since we are done with it here

    int solutionCount = 0; //var that handles total amount of solutions
found
    bool found_first = true;
    int nextrow = 0;
    int nextcol = 0;

    if (validboard(board)){ // if board is valid initially

        //this part of the code finds the first empty part in the list.

        for (int r = 0; r < SIZE; ++r) { // row for loop
            for (int c = 0; c < SIZE; ++c) { //column for loop
                if ((board[r][c])=='_') {//if the index is empty
                    nextrow = r; //keep track of row
                    nextcol = c;// keept track of column
                    found_first = true; //keeep track that an empty slot
was found
                    break; //break from column loop
                    }//end of column loop
                }
                if(found_first){ //if empty slot was found, break from
this one too
                break;
                }//end of row loop
            }//end of for loop
            if(found_first){ //if an empty slot was found, call the solve
sudoku function
                solveSudoku(board, nextrow, nextcol, solutionCount);
//find all solutions to given sudoku puzzle.
            }else{ //otherwise, the board is already full, so print the
board with one solution
                printBoard(board);
                solutionCount = solutionCount + 1;
            }
        }else{ // if not tell user and end
            std::cout << "This board configuration is invalid.\n";
```

```
            return 1;//end program
        }


    // print if no valid solution is found
    if (solutionCount == 0) {
        std::cout << "No solution found.\n";
    } else { // else tell the user total amount of solutions.
        std::cout << "Total solutions found: " << solutionCount << "\n";
    }

    return 0; //end program
}// end of main
```